

Gabarito da Lista de Exercícios: Programação Orientada a Objetos com Python

Universidade Federal do Rio Grande do Sul (UFRGS)
Faculdade de Ciências Econômicas (FCE)
Departamento de Economia e Relações Internacionais
Disciplina: Economia Computacional I
Prof. Nelson S. Santos

26 de maio de 2026

1 Bloco 1

```
1     # Quest o 1
2     class LivroTexto:
3         """
4         Atributos: titulo (str), autor (str), preco (float)
5         M todos: aplicar_desconto (modifica o pre o baseado em uma porcentagem)
6         """
7         def __init__(self, titulo, autor, preco):
8             self.titulo = titulo
9             self.autor = autor
10            self.preco = preco
11
12        def aplicar_desconto(self, porcentagem):
13            self.preco -= self.preco * (porcentagem / 100)
14
15    # Quest o 2
16    class CalculadoraSimples:
17        """
18        Atributos: ultimo_resultado (float)
19        M todos: somar, subtrair, multiplicar, dividir (todos atualizam o ltimo resultado)
20        """
21        def __init__(self):
22            self.ultimo_resultado = 0.0
23
24        def somar(self, a, b):
25            self.ultimo_resultado = a + b
26            return self.ultimo_resultado
27
28        def subtrair(self, a, b):
29            self.ultimo_resultado = a - b
30            return self.ultimo_resultado
```

```

31
32     def multiplicar(self, a, b):
33         self.ultimo_resultado = a * b
34         return self.ultimo_resultado
35
36     def dividir(self, a, b):
37         if b == 0:
38             raise ValueError("Divis o por zero n o permitida.")
39         self.ultimo_resultado = a / b
40         return self.ultimo_resultado
41
42 # Quest o 3
43 class Cofrinho:
44     """
45     Atributos: moedas (dict - mapeia o valor da moeda para a sua quantidade)
46     M todos: depositar, calcular_saldo
47     """
48     def __init__(self):
49         # Suporta moedas de 0.01, 0.05, 0.10, 0.25, 0.50 e 1.00
50         self.moedas = {0.01: 0, 0.05: 0, 0.10: 0, 0.25: 0, 0.50: 0, 1.00: 0}
51
52     def depositar(self, valor, quantidade=1):
53         if valor in self.moedas:
54             self.moedas[valor] += quantidade
55         else:
56             print("Moeda inv lida.")
57
58     def calcular_saldo(self):
59         return sum(valor * qtd for valor, qtd in self.moedas.items())
60
61 # Quest o 4
62 class ContaBancaria:
63     """
64     Atributos: titular (str), saldo (float), limite (float)
65     M todos: depositar, sacar
66     """
67     def __init__(self, titular, saldo_inicial=0.0, limite=500.0):
68         self.titular = titular
69         self.saldo = saldo_inicial
70         self.limite = limite
71
72     def depositar(self, valor):
73         if valor > 0:
74             self.saldo += valor
75
76     def sacar(self, valor):
77         if valor <= (self.saldo + self.limite):
78             self.saldo -= valor
79             return True
80         print("Saldo e limite insuficientes.")
81         return False

```

```

82
83 # Quest o 5
84 class Cronometro:
85     """
86     Atributos: tempo_decorrido (int), ativo (bool)
87     M todos: iniciar, pausar, resetar, passar_tempo
88     """
89     def __init__(self):
90         self.tempo_decorrido = 0
91         self.ativo = False
92
93     def iniciar(self):
94         self.ativo = True
95
96     def pausar(self):
97         self.ativo = False
98
99     def resetar(self):
100         self.tempo_decorrido = 0
101         self.ativo = False
102
103     def passar_tempo(self, segundos):
104         if self.ativo:
105             self.tempo_decorrido += segundos
106
107 # Quest o 6
108 class EstudanteUFRGS:
109     """
110     Atributos: nome (str), cartao (str), notas (list de floats)
111     M todos: adicionar_nota, calcular_media
112     """
113     def __init__(self, nome, cartao):
114         self.nome = nome
115         self.cartao = cartao
116         self.notas = []
117
118     def adicionar_nota(self, nota):
119         self.notas.append(nota)
120
121     def calcular_media(self):
122         if not self.notas:
123             return 0.0
124         return sum(self.notas) / len(self.notas)
125
126 # Quest o 7
127 class CarrinhoCompras:
128     """
129     Atributos: itens (list de tuplas ou dict)
130     M todos: adicionar_item, calcular_total
131     """
132     def __init__(self):

```

```

133         self.itens = [] # Lista de dicionários {'nome': str, 'preco': float}
134
135     def adicionar_item(self, nome, preco):
136         self.itens.append({'nome': nome, 'preco': preco})
137
138     def calcular_total(self):
139         return sum(item['preco'] for item in self.itens)
140
141 # Questão 8
142 class Termometro:
143     """
144     Atributos: celsius (float)
145     Métodos: atualizar_temperatura, em_fahrenheit, em_kelvin
146     """
147     def __init__(self, temperatura_inicial=0.0):
148         self.celsius = temperatura_inicial
149
150     def atualizar_temperatura(self, t):
151         self.celsius = t
152
153     def em_fahrenheit(self):
154         return (self.celsius * 9/5) + 32
155
156     def em_kelvin(self):
157         return self.celsius + 273.15
158
159 # Questão 9
160 class Retangulo:
161     """
162     Atributos: base (float), altura (float)
163     Métodos: calcular_area, calcular_perimetro
164     """
165     def __init__(self, base, altura):
166         self.base = base
167         self.altura = altura
168
169     def calcular_area(self):
170         return self.base * self.altura
171
172     def calcular_perimetro(self):
173         return 2 * (self.base + self.altura)
174
175 # Questão 10
176 class RegistroPonto:
177     """
178     Atributos: funcionario_id (str), entrada (float), saida (float)
179     Métodos: registrar_entrada, registrar_saida, calcular_horas
180     """
181     def __init__(self, funcionario_id):
182         self.funcionario_id = funcionario_id
183         self.entrada = None

```

```

184         self.saida = None
185
186     def registrar_entrada(self, hora):
187         self.entrada = hora # Formato decimal, ex: 8.5 para 08:30
188
189     def registrar_saida(self, hora):
190         self.saida = hora
191
192     def calcular_horas(self):
193         if self.entrada is not None and self.saida is not None:
194             return self.saida - self.entrada
195         return 0.0

```

2 Bloco 2

```

1     # Quest o 11
2     class RestricaoOrcamentaria:
3         """
4         Atributos: m (renda), p1 (pre o bem 1), p2 (pre o bem 2)
5         M todos: eh_viavel (verifica se p1*x1 + p2*x2 <= m)
6         """
7         def __init__(self, m, p1, p2):
8             self.m = m
9             self.p1 = p1
10            self.p2 = p2
11
12            def eh_viavel(self, x1, x2):
13                custo = (self.p1 * x1) + (self.p2 * x2)
14                return custo <= self.m
15
16    # Quest o 12
17    class UtilidadeCobbDouglas:
18        """
19        Atributos: alpha (float), beta (float)
20        M todos: calcular_utilidade, tms (Taxa Marginal de Substitui o)
21        """
22        def __init__(self, alpha, beta):
23            self.alpha = alpha
24            self.beta = beta
25
26        def calcular_utilidade(self, x, y):
27            return (x ** self.alpha) * (y ** self.beta)
28
29        def tms(self, x, y):
30            # TMS = (dU/dx) / (dU/dy) = (alpha * y) / (beta * x)
31            if x == 0:
32                return float('inf')
33            return (self.alpha * y) / (self.beta * x)
34
35    # Quest o 13

```

```

36 class EscolhaOtimap:
37     """
38     Atributos: utilidade (objeto da classe UtilidadeCobbDouglas)
39     M todos: obter_demanda
40     """
41     def __init__(self, utilidade_cobb_douglas):
42         self.u = utilidade_cobb_douglas
43
44     def obter_demanda(self, m, p1, p2):
45         # Funções de demanda analíticas clássicas Cobb-Douglas
46         soma_expoentes = self.u.alpha + self.u.beta
47         x_otimo = (self.u.alpha / soma_expoentes) * (m / p1)
48         y_otimo = (self.u.beta / soma_expoentes) * (m / p2)
49         return x_otimo, y_otimo
50
51 # Questão 14
52 class ElasticidadePreco:
53     """
54     Atributos: p1, q1, p2, q2 (valores nos momentos 1 e 2)
55     M todos: calcular_elasticidade_arco
56     """
57     def __init__(self, p1, q1, p2, q2):
58         self.p1, self.q1 = p1, q1
59         self.p2, self.q2 = p2, q2
60
61     def calcular_elasticidade_arco(self):
62         delta_q = self.q2 - self.q1
63         soma_q = self.q1 + self.q2
64         delta_p = self.p2 - self.p1
65         soma_p = self.p1 + self.p2
66
67         if delta_p == 0 or delta_q == 0:
68             return 0.0
69         # Fórmula do ponto médio (arco)
70         return (delta_q / (soma_q / 2)) / (delta_p / (soma_p / 2))
71
72 # Questão 15
73 class DemandaLinear:
74     """
75     Atributos: a (intercepto linear), b (coeficiente angular da demanda)
76     M todos: quantidade_demandada, preco_reserva
77     """
78     def __init__(self, a, b):
79         self.a = a
80         self.b = b
81
82     def quantidade_demandada(self, p):
83         q = self.a - (self.b * p)
84         return max(0.0, q)
85
86     def preco_reserva(self):

```

```

87         # Pre o onde Qd = 0
88         return self.a / self.b
89
90 # Quest o 16
91 class CustosFirma:
92     """
93     Atributos: cf (custo fixo), cv_func (função lambda ou método para custos variáveis)
94     Métodos: ct, cfme, cvme, cme
95     """
96     def __init__(self, cf, cv_func):
97         self.cf = cf
98         self.cv_func = cv_func # Deve ser uma função que aceite 'q' como argumento
99
100     def ct(self, q):
101         return self.cf + self.cv_func(q)
102
103     def cfme(self, q):
104         if q == 0: return float('inf')
105         return self.cf / q
106
107     def cvme(self, q):
108         if q == 0: return 0.0
109         return self.cv_func(q) / q
110
111     def cme(self, q):
112         if q == 0: return float('inf')
113         return self.ct(q) / q
114
115 # Quest o 17
116 class CompeticaoPerfeita:
117     """
118     Atributos: preco_mercado (float)
119     Métodos: quantidade_max_lucro (Onde  $P = CMg \rightarrow P = 2q$ )
120     """
121     def __init__(self, preco_mercado):
122         self.p = preco_mercado
123
124     def quantidade_max_lucro(self):
125         # Se  $P = 2q$ , então  $q = P / 2$ 
126         return self.p / 2.0
127
128 # Quest o 18
129 class Monopolio:
130     """
131     Atributos: c (custo marginal constante), a (intercepto demanda), b (coeficiente de demanda)
132     Métodos: maximizar_lucro (retorna q_otimo e p_otimo)
133     """
134     def __init__(self, c, a, b):
135         self.c = c
136         self.a = a
137         self.b = b

```

```

138
139     def maximizar_lucro(self):
140         # Receita Total:  $RT = a \cdot q - b \cdot q^2 \rightarrow RMg = a - 2b \cdot q$ 
141         # Condição de equilíbrio:  $RMg = CMg \rightarrow a - 2b \cdot q = c$ 
142         q_otimo = (self.a - self.c) / (2 * self.b)
143         p_otimo = self.a - (self.b * q_otimo)
144         return max(0.0, q_otimo), max(0.0, p_otimo)
145
146 # Questão 19
147 class ExcedenteEconomico:
148     """
149     Atributos: intercepto_demanda, b_demanda, intercepto_oferta, d_oferta
150     Métodos: calcular_excedentes
151     """
152     def __init__(self, a, b, c, d):
153         self.a, self.b = a, b # Demanda:  $P = a - b \cdot Q$ 
154         self.c, self.d = c, d # Oferta:  $P = c + d \cdot Q$ 
155
156     def calcular_excedentes(self):
157         # Equilíbrio:  $a - b \cdot Q = c + d \cdot Q \rightarrow a - c = (b + d) \cdot Q$ 
158         q_eq = (self.a - self.c) / (self.b + self.d)
159         p_eq = self.a - (self.b * q_eq)
160
161         # Excedente do Consumidor (área do triângulo sob a curva de demanda)
162         excedente_c = ((self.a - p_eq) * q_eq) / 2
163         # Excedente do Produtor (área do triângulo sobre a curva de oferta)
164         excedente_p = ((p_eq - self.c) * q_eq) / 2
165
166         return excedente_c, excedente_p
167
168 # Questão 20
169 class IsocustoIsoquanta:
170     """
171     Atributos: w (salário), r (custo do capital), alpha, beta (parâmetros)
172     Métodos: tmst (Taxa Marginal de Substituição Técnica)
173     """
174     def __init__(self, w, r, alpha, beta):
175         self.w = w
176         self.r = r
177         self.alpha = alpha
178         self.beta = beta
179
180     def tmst(self, K, L):
181         # Para  $Y = K^\alpha \cdot L^\beta \rightarrow PMgL = \beta \cdot Y/L; PMgK = \alpha \cdot Y/K$ 
182         #  $TMST = PMgL / PMgK = (\beta \cdot K) / (\alpha \cdot L)$ 
183         if L == 0: return float('inf')
184         return (self.beta * K) / (self.alpha * L)

```

3 Bloco 3

```

1     # Quest o 21
2     class FuncaoConsumo:
3         """
4         Atributos: c0 (consumo aut nomo), c1 (propens o marginal a consumir)
5         M todos: calcular_consumo
6         """
7         def __init__(self, c0, c1):
8             self.c0 = c0
9             self.c1 = c1
10
11        def calcular_consumo(self, y, t):
12            yd = y - t # Renda dispon vel
13            return self.c0 + (self.c1 * yd)
14
15    # Quest o 22
16    class Investimento:
17        """
18        Atributos: i0 (investimento aut nomo), b (sensibilidade aos juros)
19        M todos: calcular_investimento
20        """
21        def __init__(self, i0, b):
22            self.i0 = i0
23            self.b = b
24
25        def calcular_investimento(self, taxa_juros):
26            return self.i0 - (self.b * taxa_juros)
27
28    # Quest o 23
29    class DemandaAgregada:
30        """
31        Atributos: consumo (Objeto FuncaoConsumo), investimento (Objeto Investimento)
32        M todos: pib_equilibrio_fechada
33        """
34        def __init__(self, consumo_obj, investimento_obj, g, t):
35            self.consumo = consumo_obj
36            self.investimento = investimento_obj
37            self.g = g
38            self.t = t # Imposto fixo inicial
39
40        def pib_equilibrio_fechada(self, taxa_juros):
41            #  $Y = C_0 + C_1(Y - T) + I_0 - b*i + G$ 
42            #  $Y(1 - C_1) = C_0 - C_1*T + I_0 - b*i + G$ 
43            numerador = self.consumo.c0 - (self.consumo.c1 * self.t) + self.investimento
44            denominador = 1 - self.consumo.c1
45            return numerador / denominador
46
47    # Quest o 24
48    class MultiplicadorKeynesiano:
49        """
50        Atributos: c1 (pmc), t_aliquota (al quota tribut ria proporcional)
51        M todos: calcular_multiplicador, impacto_pib

```

```

52     """
53     def __init__(self, c1, t_aliquota):
54         self.c1 = c1
55         self.t_aliquota = t_aliquota
56
57     def calcular_multiplicador(self):
58         # Alfa = 1 / (1 - c1*(1 - t))
59         return 1 / (1 - self.c1 * (1 - self.t_aliquota))
60
61     def impacto_pib(self, delta_g):
62         return self.calcular_multiplicador() * delta_g
63
64 # Quest o 25
65 class CurvaIS:
66     """
67     Atributos: c0, c1, t, i0, b, g
68     M todos: obter_y
69     """
70     def __init__(self, c0, c1, t, i0, b, g):
71         self.c0, self.c1, self.t = c0, c1, t
72         self.i0, self.b, self.g = i0, b, g
73
74     def obter_y(self, i):
75         numerador = self.c0 - (self.c1 * self.t) + self.i0 - (self.b * i) +
76         denominador = 1 - self.c1
77         return numerador / denominador
78
79 # Quest o 26
80 class DemandaMoeda:
81     """
82     Atributos: k (sensibilidade transa o/renda), h (sensibilidade especu
83     M todos: calcular_l
84     """
85     def __init__(self, k, h):
86         self.k = k
87         self.h = h
88
89     def calcular_l(self, y, i):
90         return (self.k * y) - (self.h * i)
91
92 # Quest o 27
93 class CurvaLM:
94     """
95     Atributos: oferta_real (M/P), k, h
96     M todos: obter_i
97     """
98     def __init__(self, oferta_real, k, h):
99         self.oferta_real = oferta_real
100         self.k = k
101         self.h = h
102

```

```

103     def obter_i(self, y):
104         #  $M/P = kY - hi \rightarrow hi = kY - M/P \rightarrow i = (kY - M/P)/h$ 
105         return ((self.k * y) - self.oferta_real) / self.h
106
107 # Quest o 28
108 class EquilibrioISLM:
109     """
110     Atributos: is_obj (Objeto CurvaIS), lm_obj (Objeto CurvaLM)
111     M todos: resolver_sistema
112     """
113     def __init__(self, is_obj, lm_obj):
114         self.is_obj = is_obj
115         self.lm_obj = lm_obj
116
117     def resolver_sistema(self):
118         # Substituindo a equação da LM na IS:
119         #  $Y = (A - b*i)/(1-c1)$  onde  $i = (kY - M/P)/h$ 
120         #  $Y = (A - b*((kY - M/P)/h)) / (1-c1)$ 
121         A = self.is_obj.c0 - (self.is_obj.c1 * self.is_obj.t) + self.is_obj.t
122         prop_y = 1 - self.is_obj.c1
123
124         # Isolando Y algebricamente:
125         numerador = A + (self.is_obj.b * self.lm_obj.oferta_real / self.lm_obj.k)
126         denominador = prop_y + (self.is_obj.b * self.lm_obj.k / self.lm_obj.k)
127
128         y_est = numerador / denominador
129         i_est = self.lm_obj.get_i(y_est) if hasattr(self.lm_obj, 'get_i') else None
130         return y_est, i_est
131
132 # Quest o 29
133 class ChoqueFiscal:
134     """
135     Atributos: equilibrio_obj (Objeto EquilibrioISLM)
136     M todos: simular_aumento_g
137     """
138     def __init__(self, equilibrio_obj):
139         self.eq = equilibrio_obj
140
141     def simular_aumento_g(self, delta_g):
142         # Posição inicial
143         y0, i0 = self.eq.resolver_sistema()
144         inv0 = self.eq.is_obj.i0 - (self.eq.is_obj.b * i0)
145
146         # Choque
147         self.eq.is_obj.g += delta_g
148         y1, i1 = self.eq.resolver_sistema()
149         inv1 = self.eq.is_obj.i0 - (self.eq.is_obj.b * i1)
150
151         crowding_out = inv0 - inv1
152         return {"novo_y": y1, "novo_i": i1, "crowding_out": crowding_out}
153

```

```

154 # Quest o 30
155 class ChoqueMonetario:
156     """
157     Atributos: equilibrio_obj (Objeto EquilibrioISLM)
158     M todos: simular_expansao_monetaria
159     """
160     def __init__(self, equilibrio_obj):
161         self.eq = equilibrio_obj
162
163     def simular_expansao_monetaria(self, nova_oferta):
164         self.eq.lm_obj.oferta_real = nova_oferta
165         y_novo, i_novo = self.eq.resolver_sistema()
166         return y_novo, i_novo

```

4 Bloco 4

```

1 # Quest o 31
2 class MercadoDuopolio:
3     """
4     Atributos: a, b (Demanda:  $P = a - b(q_1 + q_2)$ ), c1, c2 (custos marginais da
5     M todos: equilibrio_cournot
6     """
7     def __init__(self, a, b, c1, c2):
8         self.a, self.b = a, b
9         self.c1, self.c2 = c1, c2
10
11     def equilibrio_cournot(self):
12         #  $q_1 = (a - c_1 - b \cdot q_2) / (2b)$  e  $q_2 = (a - c_2 - b \cdot q_1) / (2b)$ 
13         # Resolvendo o sistema linear:
14         q1 = (self.a - 2 * self.c1 + self.c2) / (3 * self.b)
15         q2 = (self.a - 2 * self.c2 + self.c1) / (3 * self.b)
16         return max(0.0, q1), max(0.0, q2)
17
18 # Quest o 32
19 class CarteiraInvestimentos:
20     """
21     Atributos: ativos (list de dicts contendo {'retorno': float, 'peso': float})
22     M todos: adicionar_ativo, calcular_retorno_esperado
23     """
24     def __init__(self):
25         self.ativos = []
26
27     def adicionar_ativo(self, retorno, peso):
28         self.ativos.append({'retorno': retorno, 'peso': peso})
29
30     def calcular_retorno_esperado(self):
31         total_pesos = sum(a['peso'] for a in self.ativos)
32         if total_pesos == 0: return 0.0
33         return sum(a['retorno'] * (a['peso'] / total_pesos) for a in self.ativos)
34

```

```

35 # Quest o 33
36 class SimuladorLeilao:
37     """
38     Atributos: lances (dict mapeando participante para o valor do lance)
39     M todos: registrar_lance, apurar_vencedor (Primeiro Pre o)
40     """
41     def __init__(self):
42         self.lances = {}
43
44     def registrar_lance(self, participante, valor):
45         self.lances[participante] = valor
46
47     def apurar_vencedor(self):
48         if not self.lances: return None, 0.0
49         vencedor = max(self.lances, key=self.lances.get)
50         preco_pago = self.lances[vencedor]
51         return vencedor, preco_pago
52
53 # Quest o 34
54 class BancoComercial:
55     """
56     Atributos: depositos (float), reserva_compulsoria_pct (float)
57     M todos: receber_deposito, simular_multiplicador
58     """
59     def __init__(self, reserva_compulsoria_pct):
60         self.depositos = 0.0
61         self.reserva_pct = reserva_compulsoria_pct / 100
62
63     def receber_deposito(self, valor):
64         self.depositos += valor
65
66     def simular_multiplicador(self, rodadas=10):
67         # M = D * (1 / reserva)
68         saldo_inicial = self.depositos
69         moeda_total = 0.0
70         potencial_emprestimo = saldo_inicial
71
72         for _ in range(rodadas):
73             moeda_total += potencial_emprestimo
74             potencial_emprestimo *= (1 - self.reserva_pct)
75
76         return moeda_total
77
78 # Quest o 35
79 class BalancaComercial:
80     """
81     Atributos: x0 (exporta o aut noma), m0 (importa o aut noma), m1
82     M todos: calcular_saldo_liquido
83     """
84     def __init__(self, x0, m0, m1):
85         self.x0 = x0

```

```

86         self.m0 = m0
87         self.m1 = m1
88
89     def calcular_saldo_liquido(self, pib_nacional, cambio_real):
90         exportacoes = self.x0 * cambio_real
91         importacoes = self.m0 + (self.m1 * pib_nacional)
92         return exportacoes - importacoes
93
94 # Quest o 36
95 class Governo:
96     """
97     Atributos: receita_tributaria (float), despesas_primarias (float), juros
98     M todos: registrar_fluxos, superavit_primario, superavit_nominal
99     """
100    def __init__(self):
101        self.receita = 0.0
102        self.gastos = 0.0
103        self.juros = 0.0
104
105    def registrar_fluxos(self, receita, gastos, juros):
106        self.receita = receita
107        self.gastos = gastos
108        self.juros = juros
109
110    def superavit_primario(self):
111        return self.receita - self.gastos
112
113    def superavit_nominal(self):
114        return self.superavit_primario() - self.juros
115
116 # Quest o 37
117 class FirmaBasica:
118     def __init__(self, q): self.q = q
119
120 class ConsumidorBasico:
121     def __init__(self, c): self.c = c
122
123 class EconomiaSimulada:
124     """
125     Atributos: firmas (list de objetos FirmaBasica), consumidores (list de c
126     M todos: adicionar_agentes, calcular_producao_agregada
127     """
128    def __init__(self):
129        self.firmas = []
130        self.consumidores = []
131
132    def adicionar_agentes(self, firma_obj, consumidor_obj):
133        if firma_obj: self.firmas.append(firma_obj)
134        if consumidor_obj: self.consumidores.append(consumidor_obj)
135
136    def calcular_producao_agregada(self):

```

```

137         return sum(f.q for f in self.firmas)
138
139 # Quest o 38
140 class IndicePrecos:
141     """
142     Atributos: cesta_padrao (dict de pesos/quantidades fixas por bem)
143     M todos: calcular_laspeyres
144     """
145     def __init__(self, cesta_padrao):
146         self.cesta = cesta_padrao # Ex: {'bem_A': 10, 'bem_B': 5}
147
148     def calcular_laspeyres(self, precos_ano_base, precos_ano_atual):
149         custo_base = sum(self.cesta[bem] * precos_ano_base[bem] for bem in s
150         custo_atual = sum(self.cesta[bem] * precos_ano_atual[bem] for bem in
151         if custo_base == 0: return 0.0
152         return (custo_atual / custo_base) * 100

```

5 Bloco 5